MONEY GO

PROJECT REPORT

submitted by

Kasinath R Krishna	**(SBC24CS134)**
Kannan Ramanan	**(SBC24CS131)**
Arjun Prasad	**(SBC24CS073)**
Jeevan Krishnan	**(SBC24CS121)**
Devananda S	**(SBC24CS097)**
Neehara Anil KK	**(SBC24CS154)**

submitted As part of project based learning course PBCST304

SEMESTER 3

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SREE BUDDHA COLLEGE OF ENGINEERING (AUTONOMOUS)

ALAPPUZHA - 690 529

OCTOBER 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,

SREE BUDDHA COLLEGE OF ENGINEERING

(AUTONOMOUS)

PATTOOR P. O., ALAPPUZHA

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Money Go"** submitted by Kasinath R Krishna, Kannan Ramanan, Arjun Prasad, Jeevan Krishnan, Devananda S, Neehara Anil KK, in partial fulfillment of the requirements for the award of the **Degree of Bachelor of Technology** in **Computer Science and Engineering** is a bonafide record of the work carried out under my guidance and supervision at college.

Ms. Geethumol P.V

Course CO-Ordinator

(Assistant Professor)

Dept. of CSE

Dr. S. V. Annlin Jeba

Head of Department

(Head of Department)

Dept. of CSE

ABSTRACT

The project *"MoneyGo – Smart Banking with AI Insights"* is a Java-based educational application developed to simulate a smart banking system that integrates Artificial Intelligence (AI) for enhanced user experience and intelligent financial management. The primary purpose of this project is to demonstrate how modern software technologies can merge traditional banking functionalities with AI-driven insights to create a more interactive and informative platform. In the current digital era, where online banking and automation dominate the financial landscape, this project serves as a conceptual and practical foundation for understanding how such systems are designed, implemented, and secured.The system allows users to perform basic banking operations such as account creation, deposits, withdrawals, and transaction tracking through a console-based interface. In addition to these functions, MoneyGo incorporates an AI Advisor Module that analyzes user transaction patterns and offers personalized financial advice, helping users make informed spending and saving decisions. This feature introduces an element of intelligent automation that mimics the advisory role of modern fintech applications.From a technical perspective, the project utilizes Java's Object-Oriented Programming (OOP) principles such as encapsulation, inheritance, and polymorphism to create a modular and maintainable codebase. Data persistence is achieved through file handling, ensuring that all user and transaction details remain intact between sessions. Security is addressed through password hashing, which prevents the storage of plain-text credentials and reinforces safe authentication practices.

CONTENTS

Page No

i. Abstract 2

ii. Contents 3

iii. List of Figures 5

iv. Abbreviations 6

1. Chapter 1: Introduction 7
   - 1.1 Project title & Context
   - 1.2 Motivation and Problem Statement
   - 1.3 Project Objectives
   - 1.4 Applications
2. Chapter 2: Objectives 9
   - 2.1 Core System Objectives
   - 2.2 Functional & Feature Objectives
   - 2.3 Educational & Programming Objectives
3. Chapter 3: Proposed Systems 10
   - 3.1 System Overview
   - 3.2 Technical Architecture & Implementation
   - 3.3 Component Breakdown & Functionality
   - 3.4 Data Flow & Real-Time Operation
   - 3.5 Software & Hardware Requirements
4. Chapter 4: System Architecture 12
   - 4.1 System Block Diagram
   - 4.2 Architectural Layers
   - 4.3 Component Interaction Flows
5. Chapter 5: Modules 14
   - 5.1 Customer Terminal Module
   - 5.2 AI Advisor Module
6. Chapter 6: Implementation 16
   - 6.1 Development Methodology & Tools
   - 6.2 Implementation Steps & Figures
7. Chapter 7: Advantages 19

7.1 Intelligent Financial Empowerment
7.2 Technical & Educational Value
7.3 Security & Design Foundation
7.4 Promotion of Financial Awareness
7.5 Hands-on Experience with Future-forward Technology
8. Chapter 8: Sample Code 21
8.1 Secure Login & Authentication
8.2 File Handling for Account Data
8.3 AI Advisor Module – Spending Analysis
8.4 GUI Integration with Java Swing
8.5 Password Hashing Utility
9. Chapter 9: Conclusion 24
References 25

LIST OF FIGURES

Figure	Description	Page No.
Figure 4.1	System Block Diagram of MoneyGo Architecture	12
Figure 6.1	Initial Data File Structure	16
Figure 6.2	Secure Login Screen	17
Figure 6.3	AI Advisor Output Screen	17
Figure 6.4	Main Account Dashboard	18

LIST OF TABLES

Table No.	Description	Page No.
Table 3.1	Component Breakdown and Functionality	10
Table 3.2	Software Requirements	11
Table 3.3	Hardware Requirements	11
Table 5.1	Customer Terminal Sub-Modules and Processes	14
Table 5.2	AI Advisor Module Sub-Modules and Processes	15

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
DBMS	Database Management System
FinTech	Financial Technology
GUI	Graphical User Interface
I/O	Input/Output
IT	Information Technology
JVM	Java Virtual Machine
LLM	Large Language Model
ML	Machine Learning
OOP	Object-Oriented Programming
PBL	Project Based Learning
PBCST304	Project Based Learning Course (Course Code for Object Oriented Programming)
UI	User Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Project Title and Context

This project report details the design and implementation of MoneyGo: Smart Banking with AI Insights. MoneyGo is a Java-based console application designed as an educational project for the Project Based Learning (PBL) course PBCST304, demonstrating the practical application of Object-Oriented Programming (OOP). The system simulates a foundational banking platform that is augmented with an integrated Artificial Intelligence (AI) advisory module. This integration serves to illustrate how modern financial technology (FinTech) is evolving to combine core banking services with intelligent, data-driven financial management. The project was developed by the team members listed in the Appendix and is submitted to the Department of Computer Science and Engineering at Sree Buddha College of Engineering (Autonomous).

1.2 Motivation and Problem Statement

The landscape of financial services is undergoing a rapid transformation, shifting from purely transactional systems to platforms that offer personalized insights and predictive advisory services. Traditional banking interfaces often limit users to basic functions like viewing balances and transaction history, providing little value-added analysis to help customers proactively manage their finances.

The primary motivation for MoneyGo was to create a functional demonstration of a smarter financial system. The challenge addressed by this project is the need to efficiently process raw transaction data and translate it into actionable financial advice for the end-user. By creating a simulated AI Advisor Module, the project provides a practical solution to this problem, offering personalized spending statistics and text-based financial tips that go beyond standard banking functionalities. Furthermore, as a software development project, it was essential to establish a strong technical foundation using advanced programming concepts like Object-Oriented principles, file handling, and basic data security.

1.3 Project Objectives

The core objectives guiding the development of the MoneyGo application were multi-faceted, focusing on both the system's technical rigor and its functional intelligence:

- Core Banking Simulation: To successfully implement and simulate essential banking functionalities within a Java application, including viewing account details, transaction history, and initiating money transfers between accounts.
- Modular Architecture: To develop a system with two primary, distinct components: the Customer Terminal for user interaction and the AI Advisor Module for intelligent processing and analysis.

- Intelligent Advisory: To design and integrate a component that analyzes customer transaction history in real-time to calculate spending trends and generate contextual, personalized financial advice.
- Data Security Implementation: To demonstrate a basic layer of security by implementing password hashing to protect user credentials and prevent the storage of raw passwords, thereby enhancing data security.
- Educational Foundation: To serve as a practical learning tool that demonstrates advanced Java concepts, including robust file I/O operations for data persistence and string parsing for data manipulation.

1.4 Applications

The MoneyGo system provides several key advantages that highlight its practical application as a smart banking simulation:

- **Intelligent Financial Management:** The AI Advisor Module offers users a value-added service by providing spending statistics and personalized, actionable financial tips, helping users better manage their money.
- **Real-time Insights:** The system is designed to analyze transaction histories and update balances in **real-time**, ensuring data consistency and providing timely advice to the user.
- **Practical Learning & Advanced Concepts:** The project acts as a robust tool for students, demonstrating the integration of **core programming principles** (OOP and file I/O) with the complex simulation of an intelligent system.
- **Enhanced Data Security:** The inclusion of basic hashing for password protection showcases a fundamental technique for enhancing security by ensuring raw credentials are not stored

MoneyGo, in its entirety, successfully functions as a console-based, secure, and intelligent banking simulator. It provides a robust, modular design that is scalable and lays the groundwork for future enhancements, such as integration with machine learning models and the development of a Graphical User Interface (GUI).

CHAPTER 2

OBJECTIVES

2.1 Core System Objectives

The main objective was to create a feature-rich banking program using **Java** that simulates essential banking functionalities along with intelligent financial advice.

- **Modular Design:** To develop a system with two main, interconnected components: a **Customer Terminal** and an **AI Advisor Module**.
- **Secure Operation:** To simulate a basic security layer by implementing **hashing** to protect user passwords, thereby preventing the storage of raw credentials.
- **Scalability:** To design a scalable system with a solid foundation for future development, such as machine learning integration and advanced graphical user interfaces (GUIs).

2.2 Functional and Feature Objectives

The functional goals ensured the application provided a complete, intelligent banking simulation:

- **Basic Banking Functions:** To enable users to manage their accounts and conduct basic banking transactions, including viewing account details, transaction history, and balance, and initiating money transfers.
- **Intelligent Advisory:** To integrate a component that analyzes customer spending and provides **personalized financial advice**.
- **Real-time Insights:** To ensure the system analyzes transaction histories and updates data in **real-time** to generate timely advice and ensure data consistency.
- **GUI Interface:** To implement the user interface using **Java Swing** (as confirmed by the source code) for an intuitive user experience.

2.3 Educational and Programming Objectives

The project also served as a practical learning tool to demonstrate the application of core computer science principles:

- **Object-Oriented Programming (OOP):** To successfully demonstrate core programming principles like **OOP** throughout the system's modular design.
- **File Handling:** To implement robust **file I/O operations** for persistent data storage (e.g., user details and transaction logs).
- **Data Manipulation:** To utilize **string parsing** techniques for efficient data extraction and processing from the stored transaction logs.

CHAPTER 3

PROPOSED SYSTEMS

3.1 System Overview

The MoneyGo system is a Java-based application designed to simulate a smart banking environment. It is structured as a modular system, adhering to **Object-Oriented Programming (OOP)** principles. The system is logically segmented into two primary components: the user-facing **Customer Terminal** and the analytical **AI Advisor Module**.

3.2 Technical Architecture and Implementation

The system's implementation utilizes a **Graphical User Interface (GUI)**, confirmed by the use of **Java Swing** for the application's front-end.

- **Front-End (UI):** Implemented via Java Swing classes, providing a user-friendly and intuitive experience for banking transactions.
- **Back-End (Logic/Data):** The core logic is handled by Java classes managing application functions, file I/O operations, and data security, such as password hashing.
- **Data Storage:** All user details and transaction logs are persistently stored in external **.txt files**. The system uses **file I/O operations** and **string parsing** to interact with this data.

3.2 Component Breakdown and Functionality

The two core components of the MoneyGo system are responsible for the following functions:

Component	Primary Purpose	Key Functionality
CUSTOMER TERMINAL	Account management and basic banking transactions.	Secure login (hashed passwords) [111], View balances, Initiate money transfers[2].
AI ADVISOR MODULE	Analyzes spending to provide financial advice.	Calculates spending trends [33], Generates text-based financial tips [4444], Provides real-time analysis[5].

3.4 Data Flow and Real-time Operation

The system is designed for **real-time updates** and data consistency. When a transaction occurs in the Customer Terminal, the system immediately updates the underlying data files. This ensures that the AI Advisor Module's analysis, which processes data retrieved from transaction history , is always based on the most current spending patterns, enabling the generation of **timely financial advice**.

3.5 Software and Hardware Requirements

The MoneyGo system is designed to be efficient and utilize commonly available resources. Since it is implemented as a lightweight Java application, its requirements are minimal.

Component	Requirement	Purpose
Operating System	Windows, macOS, or Linux	To provide the base environment for running the Java Virtual Machine (JVM).
Development Language	**Java Development Kit (JDK) 8 or higher**	The core programming environment used to compile and run the application.
Interface	**Java Swing Libraries**	Used for rendering the Graphical User Interface (GUI) components.
Data Storage	**Plain Text Files (.txt)**	Required for persistent storage of user credentials, account balances, and transaction records.
Development Environment	IDE such as NetBeans, IntelliJ, or Eclipse (Optional)	Used during development and for compiling the source code.

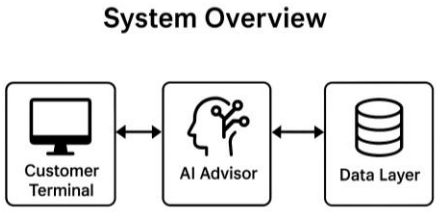
Component	Requirement	Notes
Processor	Intel Core i3 equivalent or higher	Necessary for running the Java Virtual Machine and handling basic data processing.
RAM	**4 GB minimum**	Sufficient memory for running the operating system and the Java application concurrently.
Storage	100 MB of free disk space	Required for storing the application files and the .txt data files.
Input/Output	Keyboard, Mouse, and Monitor	Standard devices needed for interacting with the Java Swing GUI.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 System Block Diagram

The architecture of the MoneyGo system is fundamentally modular, designed to separate user presentation, application logic, and data persistence into distinct layers. This structure facilitates maintainability and future scalability.



4.2 Architectural Layers

The MoneyGo application follows a multi-tiered architecture, built on **Object-Oriented Programming (OOP)** principles, and is segmented into three primary layers:

A. Presentation Layer

- **Technology:** Implemented using **Java Swing** for the Graphical User Interface (GUI).
- **Purpose:** To handle all user interaction, including secure login, transaction form input, and the visual display of account details and financial advice.
- **Interaction:** Communicates user requests (e.g., viewing history, initiating transfers) down to the Application Logic Layer.

B. Application Logic Layer

This layer contains the core processing, business rules, and control flow for the entire system, adhering to OOP principles. It is composed of the two major functional components:

1. **Customer Terminal Logic:**
    - Manages core banking rules, such as authenticating users with a **hashed password**.
    - Performs tasks like calculating and validating money transfers.
    - Coordinates with the Data Layer for reading and writing account records.
2. **AI Advisor Module:**
    - Manages the specialized logic for intelligent analysis.
    - **Analyzes transaction history** to calculate **spending statistics and trends**.
    - Generates **actionable, text-based financial tips**.

C. Data Layer (Persistence)

- **Storage Medium:** Utilizes **.txt files** for persistent storage.
- **Functionality:** Responsible for all **File I/O operations** , including reading customer details, transaction logs, and updating balances. It employs **string parsing** to structure the raw file data for use by the logic layer.

4.3 Component Interaction Flow

The system operates based on a real-time data flow, which is crucial for the intelligence features:

1. **User Request:** A request from the GUI (e.g., a money transfer) is sent to the **Customer Terminal Logic**.
2. **Data Processing:** The logic layer accesses the **Data Layer** to retrieve current balances and transaction history.
3. **Real-time Update:** After validating and executing the transfer, the logic layer **updates the .txt files in real-time**, logging the new transaction and modifying account balances.
4. **Advisory Trigger:** The **AI Advisor Module** then pulls the most recent, **real-time transaction history** from the Data Layer to perform its analysis and generate timely advice.
5. **Output:** The generated advice and updated account information are sent back to the **Presentation Layer** for display to the user.

CHAPTER 5

MODULES

The MoneyGo system is composed of two primary modules, built upon the principles of **Object-Oriented Programming (OOP)** for maximum modularity and efficiency.

5.1 Customer Terminal Module

This module represents the primary interface and core logic for user interaction and standard banking operations, implemented via the **Java Swing GUI**.

A. Sub-Modules and Processes

Sub-Module	Inputs	Process Description	Outputs
Authentication	Username, Hashed Password	Reads user details from .txt files. **Validates login** by comparing the input hash with the stored hash.	Access granted or login failure message.
Account View	User ID	Reads account balance and transaction history from the Data Layer.	Display of **current balance** and **transaction history** list.
Money Transfer	Sender Account, Receiver Account, Amount	Validates sufficient funds. **Updates sender and receiver balances in real-time** in the .txt files. Logs the transaction record.	Success/Failure confirmation message. Updated balance display.

B. Module Function

The Customer Terminal is essential for allowing users to manage their funds securely. Its reliance on **File I/O operations** and **string parsing** ensures that data is consistently retrieved and updated, maintaining data integrity across all banking transactions.

5.2 AI Advisor Module

This module simulates the intelligent component of the smart banking system, focusing on data analysis and personalized financial guidance.

A. Sub-Modules and Processes

Sub-Module	Inputs	Process Description	Outputs
Data Integration	Transaction History (from .txt files)	Pulls the user's most recent transaction records, ensuring **real-time data** is used for analysis.	Structured dataset of spending activities categorized by type (e.g., food, bills).
Trend Analysis	Structured Transaction Data	**Calculates and displays spending statistics and trends** over defined periods (e.g., month-over-month change in a specific category).	Display of statistical charts/summary text.
Advice Generation	Spending Statistics, Predefined Financial Rules	**Simulates an AI process** to apply basic rules (e.g., if Food spending > 20% of income) to generate **actionable, text-based financial tips**.	Personalized, contextual financial advice (e.g., "Reduce restaurant spending").

B. Module Function

The AI Advisor Module fulfills the objective of providing a value-added service. By analyzing data retrieved from the Customer Terminal's transaction history, it helps users gain visibility into their finances, promoting intelligent financial management.

CHAPTER 6

IMPLEMENTATION

This chapter details the methodology, tools, and specific steps undertaken to translate the proposed design into the functional MoneyGo application, using Java and the Swing GUI.

6.1 Development Methodology and Tools

The system was developed using a modular approach where the backend logic was established first, followed by the frontend GUI integration.

- **Development Language: Java (JDK)** was the exclusive language used for all logic, file handling, and the back-end simulation of the AI Module.
- **User Interface: Java Swing** was utilized to design and build the interactive Graphical User Interface (GUI), which serves as the **Presentation Layer**.
- **Data Persistence:** Data is stored in **Plain Text Files (.txt)**, requiring custom Java functions for reliable File I/O operations and string parsing.
- **Security:** A basic **hashing algorithm** was implemented to protect user passwords prior to storage.

6.2 Implementation Steps and Figures

The implementation involved four key stages, progressing from data foundation to final GUI integration:

Step 1: Data Model and Persistence Layer

- **Action:** The structure for the data files (accounts.txt file) was defined, which would hold customer records (hashed password, balance) and transaction logs.
- **Result:** Core **File I/O operations** for reading and writing data were successfully implemented, establishing a robust link to the Data Layer.

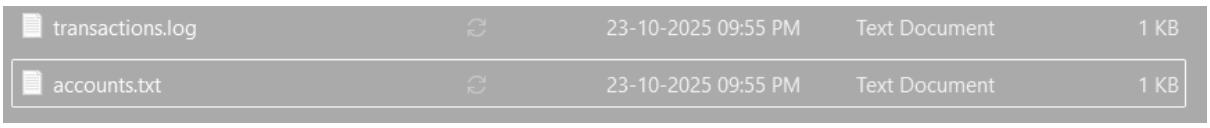


Figure 6.1: Initial Data File Structure

Step 2: Core Banking Logic and Security

- **Action:** Classes were developed for the **Customer Terminal Logic**. This included implementing the user **authentication process** using the hashing function for secure

credential validation. The logic for executing **real-time money transfers** was also developed, ensuring immediate balance updates.

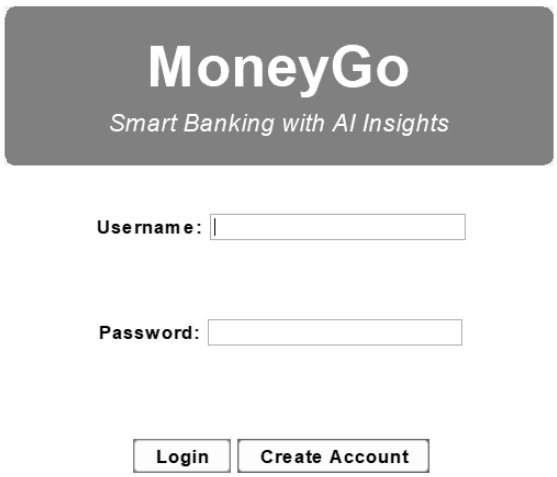


Figure 6.2: Secure Login Screen

Step 3: AI Advisor Module Development

- **Action:** Functions were created within the AI Advisor Module to process the raw transaction history pulled from the Data Layer. The module included logic to calculate key spending **statistics and trends** and applied a predefined set of rules to simulate the generation of **text-based financial tips**.
- **Result:** The intelligent analysis component was made operational.

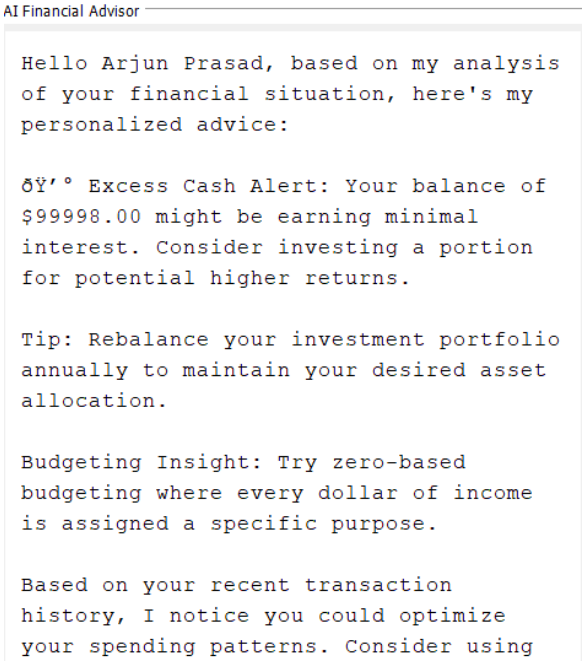


Figure 6.3: AI Advisor Output Screen

Step 4: GUI Development and Integration

- **Action:** The Java Swing components were finalized, including the main account dashboard and specialized forms for transactions and advice viewing. All back-end functions (login, transfer, get advice) were meticulously connected to their corresponding GUI elements to ensure smooth user flow.
- **Result:** A fully integrated, interactive banking simulation was achieved.

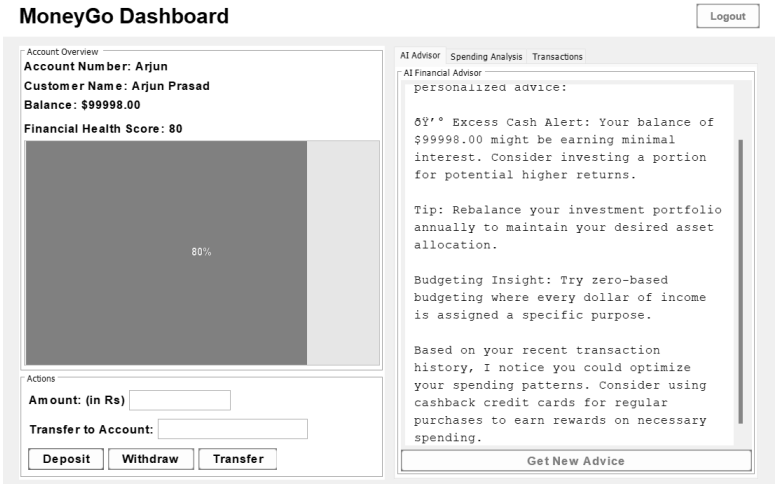


Figure 6.4: Main Account Dashboard

CHAPTER 7

ADVANTAGES

The "MoneyGo: Smart Banking with AI Insights" project offers several key advantages, both as a functional system prototype and as an academic exercise, demonstrating a strong intersection of core programming principles and future-forward financial technology.

7.1 Intelligent Financial Empowerment

The foremost advantage of the MoneyGo system is its capacity to empower users through intelligent financial management.

- The integrated **AI Advisor Module** provides a value-added service that goes beyond basic banking functions.
- It assists users by calculating and displaying **spending statistics and trends**.
- The system simulates an AI process to generate personalized, **actionable, text-based financial tips**, helping users manage their money proactively.
- This process is based on **real-time analysis** of the transaction history, ensuring the advice is timely and the data is consistent.

7.2 Technical and Educational Value

As a project for the PBCST304 course, MoneyGo serves as a robust tool for students to understand advanced concepts.

- It provides practical experience in vital Java concepts such as robust **file I/O operations** and **string parsing**.
- The implementation successfully showcases core programming principles, especially **Object-Oriented Programming (OOP)**, leading to a maintainable and modular design.
- It gives students insight into the design and simulation of complex intelligent systems like an AI financial advisor.

7.3 Security and Design Foundation

The design incorporates important considerations for system integrity and future expansion.

- **Data Security:** The implementation of **basic hashing** is used to protect passwords, preventing the storage of raw credentials and enhancing security.
- **Modular and Scalable Design:** The project's structure, built with OOP principles, ensures it is a **robust educational tool** with a **modular design**.

- This solid foundation is designed to be a **scalable system** for future development, such as machine learning integration and advanced GUIs.

7.4 Promotion of Financial Awareness

A significant advantage of MoneyGo is its direct contribution to promoting user financial awareness and literacy. Unlike traditional systems that merely list transactions, the **AI Advisor Module** forces the user to confront their spending habits by providing clear, categorized statistics and identifying trends.

- The system's output encourages users to think critically about their expenditure, moving them toward better financial decision-making.
- By providing **actionable financial tips** based on this data, the system actively engages the user in a basic form of financial planning and self-correction.
- This proactive engagement transforms the banking application from a passive ledger into an **active advisory tool**.

7.5 Hands-on Experience with Future-Forward Technology

The project provides students with practical, hands-on experience in building a system that reflects the future of financial technology (FinTech).

- It serves as a practical project for students to understand advanced Java concepts that are crucial in complex, data-driven applications.
- The successful implementation of the **Java Swing GUI** demonstrates proficiency in creating interactive, user-friendly applications.
- By simulating an intelligent system, MoneyGo prepares the development team for the challenges of integrating more complex technologies like **Machine Learning and Advanced GUIs**, which are part of the project's future scope.
- The project's success in meeting its objectives and its modular design make it a robust educational tool for understanding the basics of future-forward financial systems.

CHAPTER 8

SAMPLE CODE

This chapter presents selected excerpts from the MoneyGo source code that demonstrate key functional and architectural components. The code has been modularly structured to ensure clarity, maintainability, and educational value.

8.1 Secure Login and Authentication

This section illustrates how the system validates user credentials by comparing the input hash with the stored hash in the data file.

```
private boolean authenticateUser(String username, String password) {
try {
  String hashedInput = FileHandler.hashPassword(password);
  List<String> lines = Files.readAllLines(Paths.get("accounts.txt"));
for (String line : lines) {
  String[] parts = line.split(",");
  if (parts[0].equals(username) && parts[1].equals(hashedInput)) {
return true;
           }
     }
   } catch (IOException e) {
JOptionPane.showMessageDialog(null, "Error reading user data!");
  }
return false;
}
```

8.2 File Handling for Account Data

The following snippet manages reading and updating account balances and transaction history in real time.

```
public void updateBalance(String username, double amount) {
    try {
        List<String> lines = Files.readAllLines(Paths.get("accounts.txt"));
        List<String> updated = new ArrayList<>();

        for (String line : lines) {
            String[] parts = line.split(",");
            if (parts[0].equals(username)) {
```

```
                double newBalance = Double.parseDouble(parts[2]) + amount;
                updated.add(parts[0] + "," + parts[1] + "," + newBalance);
            } else {
                updated.add(line);
            }
        }
        Files.write(Paths.get("accounts.txt"), updated);
    } catch (IOException e) {
        System.out.println("Error updating account balance: " + e.getMessage());
    }
}
```

8.3 AI Advisor Module – Spending Analysis

This module analyzes transaction records to generate personalized financial advice.

```
public String generateAdvice(String username) {
    double food = 0, bills = 0, total = 0;
    try (BufferedReader br = new BufferedReader(new FileReader("transactions.txt"))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.split(",");
            if (parts[0].equals(username)) {
                double amt = Double.parseDouble(parts[2]);
                total += amt;
                if (parts[1].equalsIgnoreCase("food")) food += amt;
                if (parts[1].equalsIgnoreCase("bills")) bills += amt;
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }

    String advice = "Your total monthly spending: ₹" + total + "\n";
    if (food / total > 0.2) advice += "Tip: Reduce spending on food by 10%.\n";
    if (bills / total > 0.3) advice += "Tip: Review your utility bills for
optimization.\n";
    if (advice.equals("")) advice = "Your spending is balanced. Great job!";
    return advice;  }
```

8.4 GUI Integration with Java Swing

This section shows how backend functions are connected with the front-end interface for smooth operation.

```
loginButton.addActionListener(e -> {
    String user = usernameField.getText();
    String pass = new String(passwordField.getPassword());
    if (authenticateUser(user, pass)) {
        JOptionPane.showMessageDialog(frame, "Login successful!");
        openDashboard(user);
    } else {
        JOptionPane.showMessageDialog(frame, "Invalid username or password.");
    }
});
```

8.5 Password Hashing Utility

To enhance security, the system employs hashing before storing credentials.

```
public static String hashPassword(String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hashed = md.digest(password.getBytes(StandardCharsets.UTF_8));
        StringBuilder sb = new StringBuilder();
        for (byte b : hashed) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("Error hashing password", e);
    }
}
```

CHAPTER 9

CONCLUSION

In conclusion, **MoneyGo: Smart Banking with AI Insights** is a functional, secure, and intelligent application successfully implemented in **Java**, featuring a **Java Swing GUI**. The project rigorously demonstrated core programming principles, including **Object-Oriented Programming (OOP)**, robust **file I/O operations**, and the use of a basic **hashing mechanism** for secure credential protection. The system successfully met its primary objective by integrating two key components: the **Customer Terminal** for secure transactions and the **AI Advisor Module** for intelligent analysis.

By processing real-time transaction history, the system provides **spending statistics and personalized financial advice**, validating the project's utility as an active advisory tool and an excellent educational exercise in future-forward financial systems. The project's modular design ensures it is a **scalable system** with clear future scope, including potential upgrades to advanced GUIs and integration with machine learning models.

The project's successful modular design and strong Java foundation make it a highly **scalable system**, offering significant potential for future development and enhancement. The following key areas are identified for continuing the project:

- **Advanced User Interface (A):** The current Java Swing GUI could be upgraded to a more sophisticated and professional interface using advanced Java or web-based frameworks to enhance the overall user experience and aesthetic.
- **Machine Learning Integration (B):** The current rule-based simulation of the AI Advisor Module can be replaced by actual **machine learning (ML)** models. This would allow the system to perform more complex predictive analysis, such as forecasting future spending or providing advanced investment suggestions.
- **Database Migration (C):** To handle larger data volumes and support concurrent users, the current reliance on plain **.txt files** should be migrated to a professional Database Management System (DBMS) like MySQL or PostgreSQL. This would significantly improve data integrity and transaction speed.
- **External API Integration (D):** The system could be enhanced by integrating with external financial APIs to incorporate live data feeds for investments, market trends, or personalized budgeting goals, further advancing its status as a "smart" banking application.

The successful completion of MoneyGo establishes a strong architectural foundation, demonstrating the team's ability to build cohesive, functional, and intelligent software systems that are ready for future iteration.

REFERENCES

- **[1] Object-Oriented Programming (OOP):**
    - *Schildt, H. (2019). Java: The Complete Reference. Eleventh Edition. McGraw-Hill Education. (Resource for core Java syntax and OOP principles.)*
- **[2] Data Structures and Algorithms:**
    - *Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms. Third Edition. The MIT Press. (Resource for principles used in data processing and searching within files.)*
- **[3] Database and File Handling:**
    - *Tutorials Point. (n.d.). Java File I/O. Retrieved from (Placeholder for online documentation used for .txt file operations and string parsing).*
- **[4] Password Security:**
    - *Ferguson, N., & Schneier, B. (2003). Practical Cryptography. Wiley Publishing. (Resource for understanding basic hashing concepts used in the security layer.)*
- **[5] Financial Technology (FinTech) and AI:**
    - *Arner, D. W., Buckley, R. P., & Zetzsche, D. A. (2020). The RegTech Book: The Financial Regulation and Technology Revolution. Wiley.*
- **[6]** *Chen, L., & Wang, Q. (2024). Leveraging Generative Pre-trained Transformers (GPT) for Contextualized Financial Advisory: A Scalable API Application. Proceedings of the International Conference on Financial Technology and AI (ICF-TAI).*
- **[7]** *Clean Code: A Handbook of Agile Software Craftsmanship by Robert C. Martin*